1. **Write a short note on Pointer arithmetic**

Pointer arithmetic refers to performing operations on pointers that involve adding or subtracting integers or subtracting one pointer from another.

Key Points:

1. **Basic idea:**
   When you add or subtract an integer n to a pointer, the pointer moves by n elements of the type it points to (not by n bytes).
   For example, if p is an int* and sizeof(int) = 4 bytes, then p + 1 moves the pointer ahead by 4 bytes.
2. **Allowed operations:**
   - p + n or p - n: Move pointer p forward or backward by n elements.
   - p1 - p2: Difference between two pointers pointing to the same array, giving the number of elements between them.
3. **Uses:**
   Pointer arithmetic is widely used to traverse arrays or dynamically allocated memory blocks efficiently.
4. **Example:**
   int arr[5] = {10, 20, 30, 40, 50};
   int *p = arr; // points to arr[0]
   p = p + 2; // now points to arr[2], value = 30
   printf("%d", *p); // prints 30
5. **Important:**
   Pointer arithmetic is valid only within the same allocated block (e.g., within the same array). Accessing outside this range causes undefined behaviour.

2. **Allocate a two dimensional array dynamically using pointer in C language.**

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows = 3, cols = 4;
    int **arr;
    int i, j;

    // Step 1: Allocate memory for row pointers
    arr = (int **)malloc(rows * sizeof(int *));

    // Step 2: Allocate memory for each row
    for(i = 0; i < rows; i++) {
        arr[i] = (int *)malloc(cols * sizeof(int));
    }

    // Example: Initialize and print the 2D array
    for(i = 0; i < rows; i++) {
```

```
        for(j = 0; j < cols; j++) {
            arr[i][j] = i + j;  // simple initialization
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    // Step 3: Free allocated memory
    for(i = 0; i < rows; i++) {
        free(arr[i]);
    }
    free(arr);

    return 0;
}
```

3. **Differentiate between the functions malloc() and calloc()**

Feature	**malloc()**	**calloc()**
Purpose	Allocates a single block of memory	Allocates memory for an array of elements
Syntax	void *malloc(size_t size);	void *calloc(size_t n, size_t size);
Initialization	Does not initialize memory — contains garbage values	Initializes allocated memory to zero
Number of Arguments	Takes one argument (total bytes)	Takes two arguments (number of elements, size of each element)
Usage	Used when you need a block of memory	Used when you want memory for multiple elements initialized to zero
Performance	Slightly faster because it does not initialize memory	Slightly slower because it zeroes out memory

4. **Write short notes on Void pointer & Null pointer.**

Void Pointer

- A **void pointer** (void *) is a special type of pointer that can point to any data type (int, float, char, etc.).
- It is a **generic pointer** without a specific data type.
- Before dereferencing, it must be **typecast** to the appropriate pointer type.
- Useful in functions that need to handle different data types (e.g., malloc() returns a void pointer).

Example:

```
void *ptr;
int x = 10;
ptr = &x;          // void pointer points to int variable
```

```
printf("%d", *(int *)ptr);  // typecast to int* before dereferencing
```

Null Pointer

- A **null pointer** is a pointer that points to **nothing** or **no valid memory location**.
- It is usually defined as NULL (macro defined in <stdio.h> or <stdlib.h>).
- Used to initialize pointers or indicate that the pointer is not currently pointing to any valid memory.
- Helps in avoiding **dangling pointer** errors.

Example:

```
int *p = NULL;  // p points to nothing
if(p == NULL) {
   printf("Pointer is null, safe to use.");
}
```

5. **Write short notes on Dynamic memory allocation.**

Dynamic Memory Allocation

- **Dynamic memory allocation** is the process of allocating memory **at runtime** (during program execution) rather than at compile time.
- It allows programs to request memory from the heap as needed, making efficient use of memory.
- In C, dynamic memory allocation is done using library functions:
    - malloc() — allocates uninitialized memory.
    - calloc() — allocates zero-initialized memory.
    - realloc() — changes the size of previously allocated memory.
    - free() — releases allocated memory back to the system.
- Useful when the size of data structures (like arrays) is not known beforehand or varies dynamically.
- Helps manage memory efficiently, especially for large or variable-sized data.
- Important to **free** dynamically allocated memory to avoid **memory leaks**.

Example:

```
int *arr = (int *)malloc(10 * sizeof(int)); // allocate memory for 10 integers

// use the array
free(arr);  // free allocated memory
```

6. **Describe the concept of pointer to pointer with an example.**

Pointer to Pointer

- A **pointer to pointer** is a variable that stores the **address of another pointer**.
- It is useful when you need to indirectly reference a pointer or manage multiple levels of indirection.
- Declared using double asterisks (**), for example: int **ptr.
- Commonly used in dynamic memory allocation, passing pointers to functions, and handling arrays of pointers.

Example:

```
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;       // p points to x
    int **pp = &p;     // pp points to p

    printf("Value of x = %d\n", x);          // prints 10
    printf("Value via *p = %d\n", *p);       // prints 10
    printf("Value via **pp = %d\n", **pp);   // prints 10

    return 0;
}
```